

Fallstudie Textanalyse

MQM FS 2026 – Hausaufgabe 3

Sarah Inderbitzin, Andri Schneider

12.06.2026

Inhaltsverzeichnis

1	Ausgangslage und Story	1
2	Daten	2
3	Vorverarbeitung	3
4	Explorative Analyse	4
4.1	Worthäufigkeiten	4
5	TF-IDF	5
5.1	Distinktive Begriffe	5
6	Klassifikation auf Basis von Text mit Naive Bayes	5
7	Kritische Diskussion	6
8	Fazit	6
9	Anhang: Code	7

1 Ausgangslage und Story

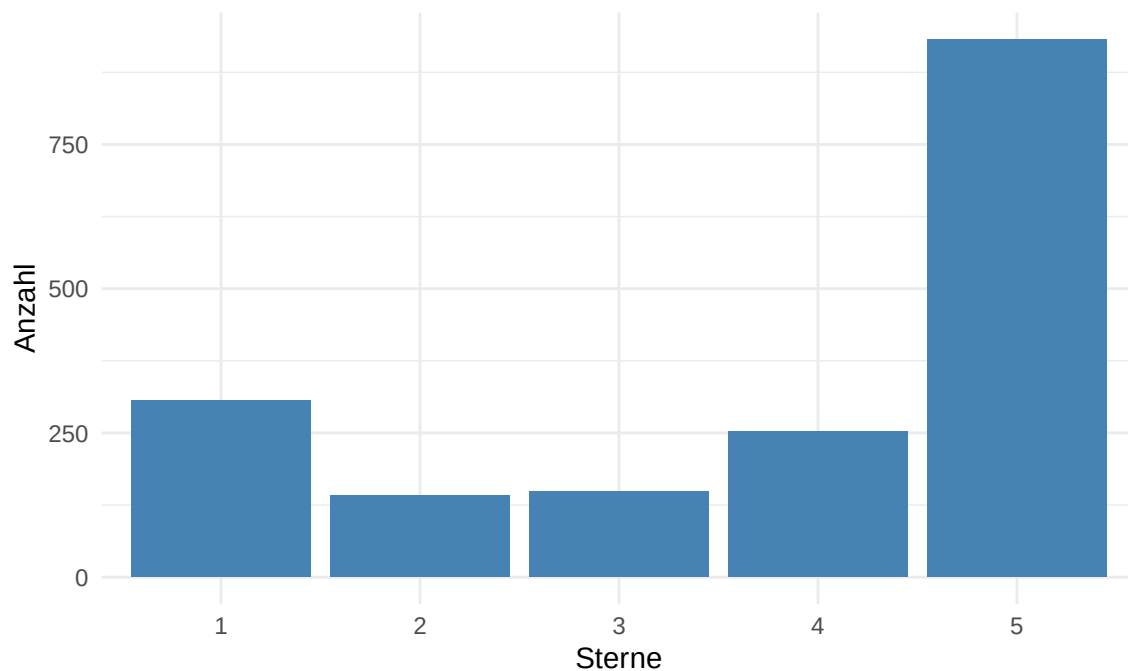
Wir geniessen als Gesellschaft heute viele Services, die den Alltag bequemer gestalten. Die Smartphones werden von einem Grossteil der Personen immer bei sich getragen und bieten unterschiedliche Möglichkeiten für solche Services; Vom Geniessen verschiedenster Musik per Knopfdruck, über die Konnektivität mit Familie, Freunden, Arbeit und gefühlt dem Rest der Welt, bis hin zum Ersatz des Portemonnaies durch Digital-Wallet-Möglichkeiten von beispielsweise ID, Krankenkassenkarte und auch Zahlungsmöglichkeiten ist alles in dem unscheinbaren Handy enthalten. Ob das wirklich so toll ist und nur Positiven birgt, müssen alle für sich selbst bestimmen. Deswegen können digitale Zahlungsmittel wie TWINT Fluch und Segen zugleich sein. TWINT ist seit dessen Einführung für viele im Alltag zur Normalität geworden; Ultra praktisch muss kein Bargeld und keine EC-Karte zum einzukaufen mitgenommen werden und es kann auch mal spontan irgendwo etwas gekauft werden. Steht man aber an der Kasse und hat keinen Akku, kein Netz oder die App selbst hat ein Problem, ist das dann doch ziemlich ungünstig.

Somit ist das Potenzial für verärgerte Kundschaft nicht klein, welche dann teilweise auch ein Ventil für ihren Frust sucht. Dieses Ventil kann eine Rezension sein. Somit sind in den Rezensionen der Apps immer wieder geladene Texte der Kundschaft zu finden, bei der die App “im dümmsten Moment” nicht funktioniert. Auch wenn die App an sich grösstenteils sehr gut funktioniert und das auch in den Rezensionen zu lesen ist, sind genau solche Aussetzer wichtig zu erkennen.

Genau hier setzt unsere Analyse an. Eine einzelne Sternebewertung sagt zwar, ob jemand zufrieden war, aber nicht, warum. Wir fragen deshalb: Lassen sich aus den Texten der TWINT-Rezensionen systematisch jene Themen herauslesen, die hinter negativen Bewertungen stehen und unterscheiden sich diese sprachlich klar von den positiven? Wir erwarten erstens eine insgesamt eher positive bis polarisierte Bewertungsverteilung, wie sie für etablierte Alltags-Apps typisch ist. Zweitens vermuten wir, dass sich negative Reviews weniger auf die Idee der App sondern eher auf konkrete Ausfälle beziehen; Begriffe rund um Login, Verbindung oder Zahlungsabbrüche dürften in den schlecht bewerteten Texten überdurchschnittlich häufig auftauchen. Ob sich diese Erwartung bestätigt und ob ein einfaches, interpretierbares Modell positive von negativen Reviews allein anhand des Textes trennen kann, prüfen wir in den nächsten Minuten.

2 Daten

```
## Anzahl Texte:      1781
## Leere Texte:       0
## Anteil positiv:    0.67
## Wörter im Mittel:  17.38
## Anzahl Sprachen und deren Verteilung:
##
##      af    da    de    en    fr    it    nl    no    sv
## 123     4     2 1608    31     3     5     1     1     3
```



Der Datensatz umfasst 1'781 Reviews und keines davon ist leer. Die Reviews sind mit durchschnittlich 17 Wörtern eher kurz, was für App-Bewertungen aber typisch ist.

Die Rating-Labels sind folgendermassen verteilt: 67 % positiv (4-5 Sterne), 33 % negativ (1-3 Sterne). In der Grafik ist ersichtlich, dass die 5-Sternebewertung am häufigsten gegeben wird, gefolgt von einem Stern. Die restlichen Bewertungen (2-4) sind relativ gleichmässig verteilt im Mittelmass. Es sind mehrheitlich positive Bewertungen, aber es gibt genug negatives Material (rund ein Drittel), um Muster hinter schlechten Bewertungen herauszuarbeiten.

Circa 90% der Reviews wurden als Deutsch erkannt. Bei der Übersetzung wurde vom Google-Translator Englisch (31x), Französisch (3x) und Italienisch (3x) korrekt erkannt und ins deutsche übersetzt. Die restlichen erkannten Sprachen (Niederländisch, Dänisch, Französisch, Schwedisch, Norwegisch und Afrikanisch) sind bei genauerer Betrachtung falsch klassifiziert, da diese bereits im Original Deutsch waren; sie wurden trotzdem “übersetzt”. Zudem konnten einige Texte nicht erkannt werden (NaN). Diese waren entweder Hochdeutsch mit Schreibfehlern oder verhältnismässig vielen englischen Begriffen oder auf Schweizerdeutsch verfasst. -> hier zeigen der Beispiele in R Data Explorer

Folgend werden nur noch die originalen oder auf Deutsch übersetzten Texte verwendet. Aufgrund der geringen Anzahl der Schweizerdeutschen Texte werden diese nicht als entscheidend betrachtet und deswegen nicht noch separat übersetzt.

3 Vorverarbeitung

```
## doc_id
## 1      1
## 2      2
## 3      3
## 4      4
## 5      5
## 6      6
##
## 1
## 2
## 3
## 4
## 5
## 6 Das neue Update ist ganz schlecht. Zum bezahlen wurde der QR leser vergrößert und man kann fast n
## rating label date
## 1      5 positiv 2026-06-03 09:01:41
## 2      1 negativ 2026-05-30 19:47:39
## 3      5 positiv 2026-05-27 10:34:52
## 4      1 negativ 2026-05-26 09:20:01
## 5      1 negativ 2026-05-26 08:03:24
## 6      1 negativ 2026-05-24 14:49:15
```

Wir normalisieren die Tokens mit Lemmatisierung statt Stemming, da die deutsche Sprache bekannterweise komplex ist. Der Snowball-Stemmer schneidet Endungen rein regelbasiert ab und scheitert an unregelmässigen Verben wie zum Beispiel gehen wo ging und gegangen nicht korrekt gefunden werden würden. Zudem scheitert Stemming an Umlautwechseln, und es erzeugt häufig keine echten Worte, die nicht zu verstehen sind. Da unsere Analyse explizit interpretierbar sein soll und die distinktiven Begriffe aus TF-IDF wie auch die späteren Topics lesbar bleiben müssen, führen wir die Wörter mit **udpipe** und dem Modell **german-gsd** auf ihre echte Grundform zurück. **udpipe** übernimmt dabei Tokenisierung, Wortarterkennung (POS) und Lemmatisierung in einem Schritt, weshalb die separate Tokenisierung über **unnest_tokens** entfällt.

Satzzeichen, Symbole und Zahlen entfernen wir über die Wortart (**upos**) statt über eine Zeichenregel; das ist präziser als **strip_numeric**. Zahlen tragen in App-Reviews kaum inhaltliche Bedeutung (Sternangaben, Datums- und Versionsnummern) und fallen deshalb weg. Die Stoppwörter entfernen wir bewusst auf der Lemma-Ebene mit der Snowball-Liste für Deutsch: Da flektierte Funktionswörter wie “ist”, “war” oder “sind” alle auf das Lemma “sein” zurückgeführt werden, greift die Liste auf der Grundform zuverlässiger als auf der Oberflächenform. Dies ist ein konkreter Vorteil der Lemmatisierung. Findet **udpipe** für ein Token kein Lemma (etwa bei Tippfehlern, englischen Begriffen oder Schweizerdeutsch), behalten wir die Oberflächenform, damit kein Token verloren geht.

Produktnamen wie TWINT, UBS oder App behalten wir bewusst, da sie für die Domäne zentral sind und

in der Leitfrage eine Rolle spielen. Eine Filterung seltener Wörter nehmen wir an dieser Stelle nicht global vor: Sie erfolgt erst beim Aufbau der Feature-Matrix und ausschliesslich auf den Trainingsdaten, damit keine Testinformation einfließt.

4 Explorative Analyse

4.1 Worthäufigkeiten

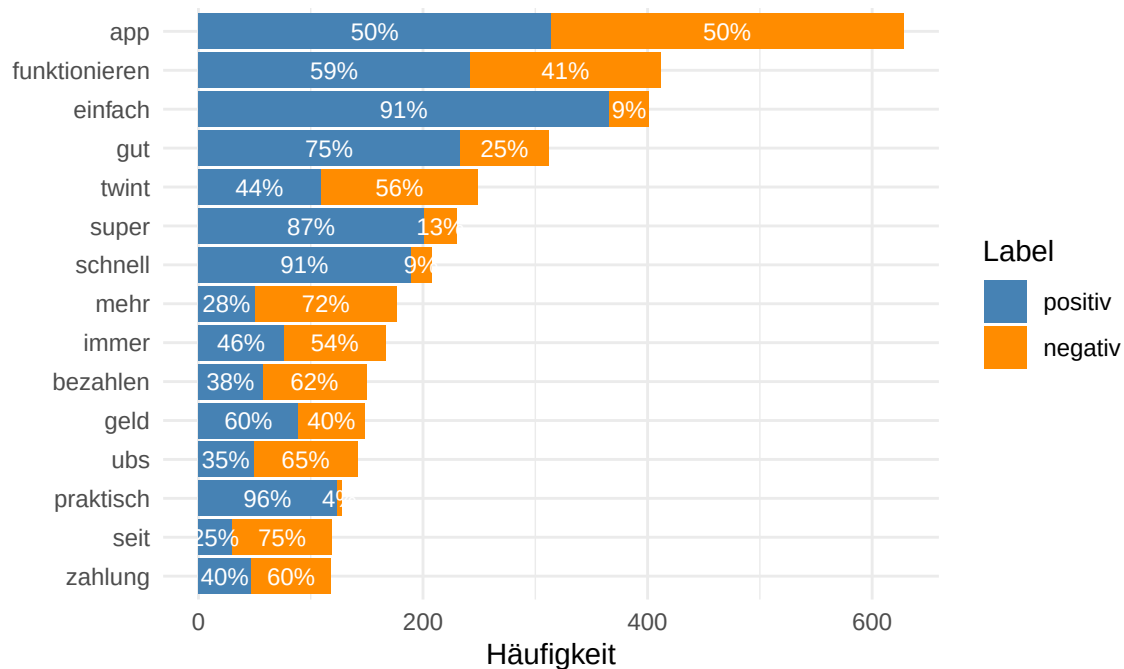


Abbildung 1: Häufigste Stems im Korpus

Die häufigsten Lemmas bestätigen die erwartete Trennung bereits explorativ. Bedienbarkeitsbegriffe wie **einfach**, **super**, **schnell** und **praktisch** treten weit überdurchschnittlich in positiven Reviews auf, während sich Wörter rund um den Zahlungsvorgang (**zahlung**, **bezahlen**) sowie **ubs**, **seit** und **mehr** in den negativen Reviews konzentrieren. Zu beachten ist der Anteil von 67% positiven Reviews: Erst Wörter klar oberhalb dieser Schwelle sind tatsächlich positiv assoziiert. So liegt **app** mit 50% unter der Basis und ist damit eher negativ konnotiert. Sehr häufige, Wörter wie **funktionieren** (59%) tragen dagegen kaum zur Unterscheidung bei, vermutlich durch Formulierungen wie “funktioniert gut” oder “funktioniert nicht”. Frage Sarah: wieso sind nur alle mit mehr als 67% tatsächlich positiv?

5 TF-IDF

5.1 Distinktive Begriffe

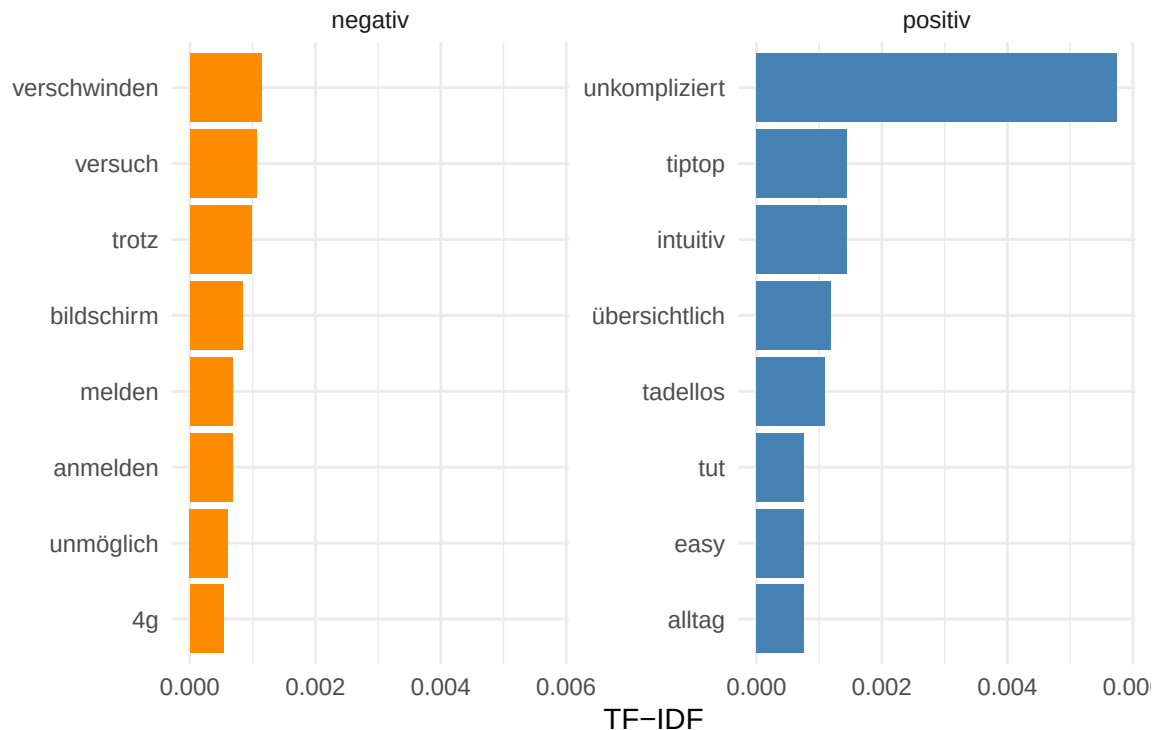


Abbildung 2: Top TF-IDF Stems je Klasse

Die klassenweise TF-IDF-Analyse bestätigt die erwartete Trennung deutlich. Auf der negativen Seite dominieren konkrete Fehlerorte: **anmelden** und **melden** verweisen höchstwahrscheinlich auf Login-Probleme, **4g** auf die Netzabhängigkeit, **bildschirm** und **verschwinden** vermutlich auf eingefrorene Anzeigen bzw. fehlgeschlagene Zahlungen, begleitet von **versuch** und **unmöglich**. Die positiven Reviews heben dagegen die Bedienbarkeit und den Alltagsnutzen hervor: **unkompliziert**, **intuitiv**, **übersichtlich**, **tadellos** und **easy**. Damit zeigt sich genau das vermutete Muster: Negative Bewertungen beziehen sich auf konkrete Ausfälle, positive auf die Einfachheit der App. Anders als im reinen Häufigkeitsplot verschwindet das sehr häufige, aber label-neutrale **app**, was den Nutzen der TF-IDF-Gewichtung illustriert.

Zu beachten ist, dass TF-IDF bei nur zwei Klassen vor allem klassen-exklusive Begriffe betont; einzelne davon können selten sein, weshalb die distinktiven Wörter nicht mit den wichtigsten Themen gleichzusetzen sind. Eine Prüfung der absoluten Häufigkeiten zeigt hier allerdings, dass die distinktiven negativen Begriffe substantiell auftreten (7- bis 15-mal) und damit keine zufälligen Einzelfälle sind. Frage sarah: was ist da gemeint mit klassen-exklusiven Begriffen und was wären Beispiele dafür?

6 Klassifikation auf Basis von Text mit Naive Bayes

Für den Naive-Bayes braucht es einen train/test-split, damit die Evaluation auf unabhängigen Daten erfolgt. Die Wortliste für die Bag of Words-Features wird ausschließlich aus den Trainingsdaten erstellt, um keine Testinformation einfließen zu lassen. Die muss gemacht werden, damit die Performance nicht künstlich aufgebläht wird.

[1] 513

```

## Accuracy:                0.76
## Mehrheits-Baseline:      0.66
## Konfusionsmatrix (Zeilen = Vorhersage, Spalten = tatsächlich):
##
##           Tatsächlich
## Vorhersage negativ positiv
##   negativ      61      27
##   positiv      59     210
##
## Precision / Recall / F1 je Klasse:
##
## Klasse Precision Recall   F1
## negativ      0.693  0.508 0.587
## positiv      0.781  0.886 0.830

```

Naive Bayes erreicht auf den Testdaten eine Accuracy von 0.76 und übertrifft damit die Mehrheits-Baseline von 0.66. Die Mehrheitsbaseline ist berechnet, wenn man für alle Beobachtungen einfach “positiv” sagt, wie hoch wäre dann die Accuracy. Das Modell lernt also tatsächlich textbasierte Muster und stuft nicht random ein. Aufschlussreicher als die Gesamtgenauigkeit ist jedoch der Blick der jeweiligen Klassen: Während positive Reviews zuverlässig erkannt werden (Recall 0.88), liegt der Recall der negativen Klasse nur bei 0.5. Von 120 tatsächlich negativen Reviews werden 59 fälschlich als positiv eingestuft. Die Precision der negativen Klasse ist mit 0.69 hingegen brauchbar: Was das Modell als negativ markiert, ist meist auch negativ. Für die Leitfrage bedeutet das, dass sich negative Reviews zwar überzufällig am Text erkennen lassen, ein einfaches BoW-Modell aber rund die Hälfte der Beschwerden übersieht. Gründe dafür könnten sein die unausgeglichene Klassenverteilung, die Kürze der Texte und die Unabhängigkeitsannahme des Verfahrens, die Negationen wie „nicht schlecht“ auseinanderreißt.

7 Kritische Diskussion

- **Grenzen BoW/TF-IDF (W12, S.5):** ignorieren Wortreihenfolge, Kontext, Mehrdeutigkeit, semantische Ähnlichkeit (“nicht schlecht” vs. “schlecht”).
- **Sentiment-Wortlisten (W12, S.22):** Negation, Ironie, kontextabhängige Wörter, domänenspezifische Begriffe; Liste ist nie neutral.
- **Preprocessing (W11, S.38):** Stemming kann Bedeutung verzerren; seltene Wörter können Rauschen oder wichtige Hinweise sein.
- **Daten & Bias (W11, S.25):** Korpus-Verzerrung, Selektionsbias bei Reviews, Rechtschreibfehler (W11, S.51), Sprache/Übersetzung (W12, S.26).
- **Methodische Sauberkeit (W11, S.45):** Wortliste nur auf Trainingsdaten.

8 Fazit

Kurzer Schluss (3–5 Sätze): Antwort auf die Leitfrage + was ihr mit mehr Zeit/Daten anders machen würdet.

9 Anhang: Code

```
# --- Kern (läuft auf dem Demo-Datensatz durch) ---
library(tidyverse) # dplyr, ggplot2, stringr, tidyr
library(tidytext)  # unnest_tokens, bind_tf_idf, cast_dtm, tidy()
library(tokenizers) # tokenize_words / tokenize_ngrams (im Kurs verwendet)
library(stopwords) # stopwords(source="snowball", language="de")
library(SnowballC) # wordStem(..., language="de") -> Stemming
library(naivebayes) # naive_bayes() -> Klassifikation auf BoW (Kurs-Beispiel)
library(knitr)
library(dplyr)
library(udpipe)
library(topicmodels)
# --- Datensatz: UBS-Twint App
reviews <- read.csv("C:/Users/InderbitzinSarah/Repos/MQM_HA_3/twint_reviews_de.csv")
# names(reviews)
# "doc_id", "app", "text", "rating", "date", "thumbsUpCount", "reviewCreatedVersion", "lang", "text_de"

## Transformationen
# Rating -> binäres Label
reviews <- reviews |>
  mutate(label = factor(if_else(rating >= 4, "positiv", "negativ"),
                        levels = c("negativ", "positiv")))

cat("Anzahl Texte:      ", nrow(reviews), "\n")
cat("Leere Texte:      ", sum(is.na(reviews$text) | str_trim(reviews$text) == ""), "\n")
cat("Anteil positiv:    ", round(mean(reviews$label == "positiv"), 2), "\n")
cat("Wörter im Mittel:  ", round(mean(str_count(reviews$text, "\\w+"), na.rm = TRUE), 2), "\n\n")

cat("Anzahl Sprachen und deren Verteilung:")
print(table(reviews$lang, useNA = "ifany"))

# Verteilung Reviews
ggplot(reviews, aes(factor(rating))) +
  geom_bar(fill = "steelblue") +
  labs(x = "Sterne", y = "Anzahl") + theme_minimal()

#print(reviews[as.character(reviews$lang) != "de", c("lang", "text")])

#Umbenennung und Selektion
reviews <- reviews |> select(doc_id, text = text_de, rating, label, date)
head(reviews)

de_stop <- stopwords(source = "snowball", language = "de")
# snowball genommen da es gut für Basismodelle ist

# --- udpipe-Modell laden (einmalig herunterladen) -----
# Der Download legt eine .udpipe-Datei im Projektordner ab. Danach wird nur
# noch geladen -> beim Knitten nicht jedes Mal neu herunterladen.
ud_files <- list.files(pattern = "german-gsd.*\\.udpipe$")
if (length(ud_files) == 0) {
```

```

model_file <- udpipe_download_model(language = "german-gsd")$file_model
} else {
  model_file <- ud_files[1]
}
ud_model <- udpipe_load_model(model_file)

# --- Annotation: Tokenisierung + POS + Lemma in einem Schritt -----
anno <- udpipe_annotate(
  ud_model,
  x = reviews$text,
  doc_id = as.character(reviews$doc_id),
  parser = "none" #kein Dependency-Parsing nötig -> deutlich schneller.
) |>
  as.data.frame()

# --- Tokens aufbereiten -----
tokens <- anno |>
  filter(!upos %in% c("PUNCT", "SYM", "NUM", "X")) |> # Satzzeichen/Zahlen/Symbole raus
  # Optional (einkommentieren): nur Inhaltswörter behalten ->
  # lesbarere TF-IDF-Begriffe und Topics:
  # filter(upos %in% c("NOUN", "PROPN", "VERB", "ADJ", "ADV")) |>
  mutate(
    word = str_to_lower(token),
    lemma = str_to_lower(coalesce(lemma, token)) # wenn kein Lemma vorhanden ist, nimmt es den
  ) |>
  filter(!lemma %in% de_stop) |> # Stoppwörter auf Lemma-Ebene
  filter(str_detect(lemma, "[a-zäöüß]")) |> # nur Lemmas nehmen, die auch tatsächlich Buchstaben sind
  left_join(reviews |> transmute(doc_id = as.character(doc_id), label),
    by = "doc_id") |>
  select(doc_id, label, word, lemma)

#nun wurden alle reviews in token (wörter) aufgesplittet und jedes wort hat das label positiv oder negativ
# Top 15 Lemmas nach GESAMTHäufigkeit bestimmen
top_n_lemma <- tokens |>
  count(lemma, sort = TRUE) |>
  slice_head(n = 15)

# Häufigkeiten je Lemma UND Label, nur für diese Top-15
top_lemma <- tokens |>
  filter(lemma %in% top_n_lemma$lemma) |> # nur die Top-15 Lemmas betrachten
  count(lemma, label) |> # Häufigkeit zählen für Lemma und Label
  mutate(label = factor(label, levels = c("positiv", "negativ"))) |> # Label-Reihenfolge für konsistent
  group_by(lemma) |>
  mutate(pct = n / sum(n)) |> # Anteil innerhalb des Balkens
  ungroup()

ggplot(top_lemma, aes(x = n, y = fct_reorder(lemma, n, .fun = sum), fill = label)) +
  geom_col(position = position_stack(reverse = TRUE)) +
  geom_text(aes(label = scales::percent(pct, accuracy = 1)),
    position = position_stack(vjust = 0.5, reverse = TRUE),
    color = "white", size = 3) +
  scale_fill_manual(values = c(positiv = "steelblue", negativ = "darkorange")) +
  labs(x = "Häufigkeit", y = NULL, fill = "Label") +

```



```

theme_minimal()
# Hier dient die KLASSE als "Dokument", um charakteristische Wörter zu finden.
tfidf_klasse <- tokens |>
  count(label, lemma, sort = TRUE) |>
  bind_tf_idf(lemma, label, n) |>
  group_by(label) |>
  slice_max(tf_idf, n = 8, with_ties = FALSE) |>
  ungroup()

ggplot(tfidf_klasse,
       aes(tf_idf, reorder_within(lemma, tf_idf, label), fill = label)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ label, scales = "free_y") +
  scale_y_reordered() +
  scale_fill_manual(values = c(positiv = "steelblue", negativ = "darkorange")) +
  labs(x = "TF-IDF", y = NULL) +
  theme_minimal()

library(rsample)
library(gam)
set.seed(123)

doc_ids <- as.character(reviews$doc_id)
intrain <- initial_split(reviews, prop=0.8, strata=label) # stratifizierte Aufteilung, damit die Klass
train_ids <- training(intrain)
test_ids <- testing(intrain)

train_ids_old <- sample(doc_ids, size = floor(0.7 * length(doc_ids)))
test_ids_old <- setdiff(doc_ids, train_ids)
# Wortliste NUR aus Trainings-Dokumenten.
min_count <- 5 # Mindesthäufigkeit im Training -> Entscheidung, siehe Text

word_list <- tokens |>
  filter(doc_id %in% train_ids$doc_id) |>
  count(lemma, sort = TRUE) |>
  filter(n >= min_count) |>
  pull(lemma)

length(word_list) # Anzahl Features kurz prüfen
# Baut die BoW-Matrix aus der bestehenden tokens-Annotation für ein doc_id-Set.
build_features <- function(doc_ids, word_list) {
  feats <- tokens |>
    filter(doc_id %in% doc_ids, lemma %in% word_list) |>
    count(doc_id, lemma) |>
    pivot_wider(names_from = lemma, values_from = n, values_fill = 0)

  miss <- setdiff(word_list, names(feats)) # im Subset nicht vorkommende Lemmas
  feats[miss] <- 0

  # Label zurückholen; Dokumente ohne Feature-Wort als Null-Zeile behalten
  reviews |>
    transmute(doc_id = as.character(doc_id), label) |>

```

```

    filter(doc_id %in% doc_ids) |>
    left_join(feats, by = "doc_id") |>
    mutate(across(all_of(word_list), ~ replace_na(.x, 0))) |>
    select(doc_id, label, all_of(word_list))
}

# Für Naive Bayes: nur ob ein Wort vorkommt (binär, W11 S.44)
to_binary <- function(feats_df, word_list) {
  feats_df |>
    mutate(across(all_of(word_list),
      ~ factor(.x > 0, levels = c(FALSE, TRUE),
        labels = c("no", "yes"))))
}

train_bin <- to_binary(build_features(train_ids$doc_id, word_list), word_list) |> select(-doc_id)
test_bin <- to_binary(build_features(test_ids$doc_id, word_list), word_list) |> select(-doc_id)

nb <- naive_bayes(label ~ ., data = train_bin, laplace = 1) # Laplace-Glättung
pred <- predict(nb, newdata = test_bin |> select(-label))

konfusion <- table(Vorhersage = pred, Tatsächlich = test_bin$label)
accuracy <- mean(pred == test_bin$label)

# Kennzahlen je Klasse aus der Konfusionsmatrix (dependency-frei)
metriken <- purrr::map_dfr(levels(test_bin$label), function(kl) {
  tp <- konfusion[kl, kl]
  fp <- sum(konfusion[kl, ]) - tp
  fn <- sum(konfusion[, kl]) - tp
  precision <- tp / (tp + fp)
  recall <- tp / (tp + fn)
  tibble(Klasse = kl, Precision = precision, Recall = recall,
    F1 = 2 * precision * recall / (precision + recall))
})

baseline <- max(prop.table(table(test_bin$label))) # Mehrheitsklassen-Accuracy
cat(sprintf("Accuracy: %.2f\n", accuracy))
cat(sprintf("Mehrheits-Baseline: %.2f\n", baseline))

cat("Konfusionsmatrix (Zeilen = Vorhersage, Spalten = tatsächlich):\n")
print(konfusion)

cat("\nPrecision / Recall / F1 je Klasse:\n")
print(as.data.frame(metriken), row.names = FALSE, digits = 3)
# SentiWS einlesen (https://wortschatz.uni-leipzig.de/de/download/SentiWS) -> ERSETZEN.
# Format je Zeile: "Wort|POS \t value \t flexionen"
# senti_lex <- read_delim(c("SentiWS_v2.0_Positive.txt", "SentiWS_v2.0_Negative.txt"),
#   delim="\t", col_names=c("wort_pos", "value", "flex")) |>
#   mutate(word = str_to_lower(str_remove(wort_pos, "\\|.*"))) |>
#   select(word, value)

# --- Mini-Seed-Lexikon (PLATZHALTER, damit die Vorlage läuft) ---
senti_lex <- tibble(
  word = c("hervorragend", "freundlich", "tolle", "leckere", "empfehlenswert", "gut", "gerne",
    "gemütlich", "zuvorkommend", "fantastisch", "traum", "frische", "kreative", "top",

```

```

      "super", "freundliches", "aufmerksamer", "ausgezeichnete", "solides",
      "enttäuschend", "teuer", "mangelhaft", "kalt", "unfreundliches", "klein", "hohen",
      "katastrophaler", "überteuert", "fade", "laut", "lauwarm", "schlechte", "schlecht", "lang"),
  value = c(rep(1, 19), rep(-1, 15))
)

sentiment_scores <- reviews |>
  select(doc_id, label, rating, text) |>
  unnest_tokens(word, text) |>
  inner_join(senti_lex, by = "word") |>
  summarise(sentiment = sum(value), .by = c(doc_id, label, rating))

```